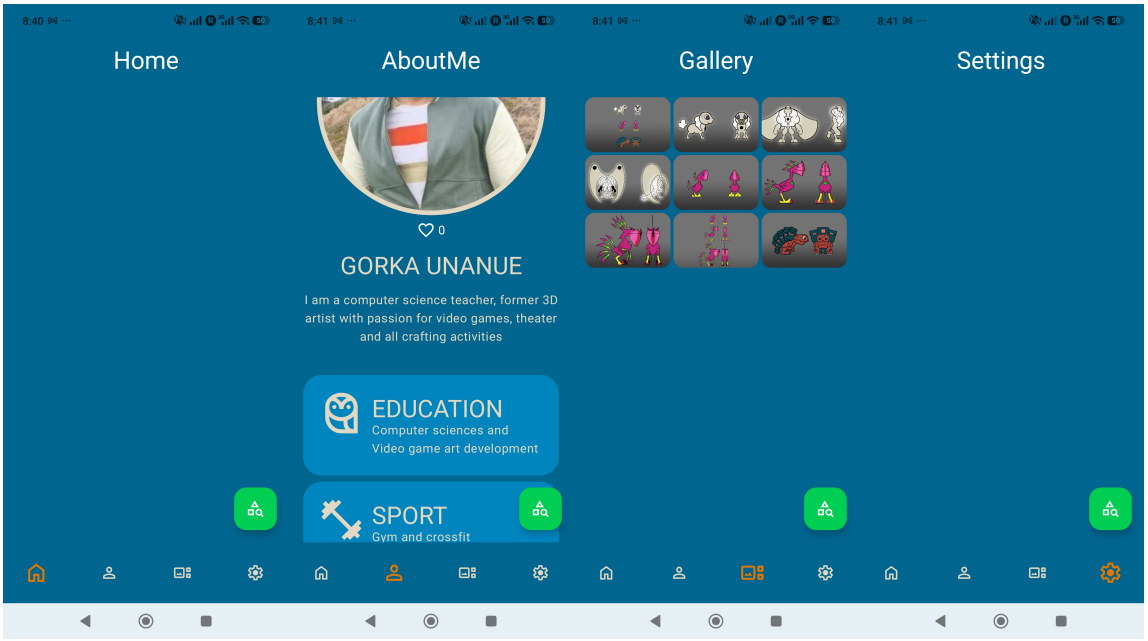
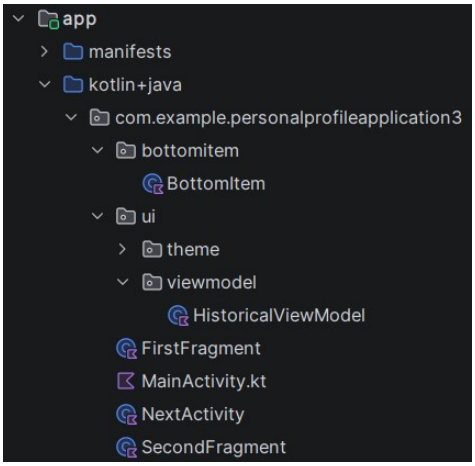
UD3_Ejercicios

Ejercicio01. Implementa un menú con 4 puntos:

- Home
- About me
- Galería
- Settings

El objetivo es implementar el menú y su funcionalidad, los composables de Home y Settings pueden estar vacías. About me será el composable de los temas anteriores y en galería mostraremos una galería de imágenes con un LazyGrid.





MainActivity.kt | HistoricalViewModel.kt | BottomItem.kt | str

```kotlin
package com.example.personalprofileapplication3.ui.viewmodel

import ...

class HistoricalViewModel: ViewModel() {
    1 Usage
    val historical = mutableStateListOf<String>()

    2 Usages
    var likes by mutableIntStateOf( value = 0)
        private set

    fun register (message: String){
        historical.add(message)
    }

    fun addLike() {
        likes++
    }

    fun getlikes():Int {
        return likes
    }
}
```

MainActivity.kt | HistoricalViewModel.kt | BottomItem.kt | </>

```kotlin
package com.example.personalprofileapplication3.bottomitem

6 Usages
data class BottomItem (
    val route: String,
    val icon: Int,
    val label: String
)
```

MainActivity.kt | HistoricalViewModel.kt | BottomItem.kt | strings.xml

currentRoute 1/4

```kotlin
fun ViewContainer(navController: NavHostController) {
    val snackbarHostState = remember { SnackbarHostState() }
    val scope = rememberCoroutineScope()
    val historial = remember { mutableStateListOf<String>() }
    val bottomItems = listOf(
        BottomItem( route = "home", icon = R.drawable.ic_home, label = "Home"),
        BottomItem( route = "AboutMe", icon = R.drawable.ic_aboutme, label = "AboutMe"),
        BottomItem( route = "Gallery", icon = R.drawable.ic_gallery, label = "Gallery"),
        BottomItem( route = "Settings", icon = R.drawable.ic_settings, label = "Settings")
    )
    var topBarTitle by remember { mutableStateOf( value = "Home") }
    PersonalProfileApplicationTheme {
        Scaffold(
            snackbarHost = { SnackbarHost(snackbarHostState) },
            topBar = { Topbar(topBarTitle) },
            bottomBar = {
                BottomBar(
                    navController = navController,
                    bottomItems = bottomItems,
                    topBarTitle = { newTitle -> topBarTitle = newTitle }
                )
            },
            content = { padding ->
                NavHost(
                    navController = navController,
                    startDestination = "home",
                    modifier = Modifier.padding( paddingValues = padding)
                ) {
                    composable( route = "home") { ContentHome(historial, onRegister = { historial.add(it) }) }
                    composable( route = "AboutMe") {
                        ContentAboutMe(
                            historial,
                            onRegister = { historial.add(it) })

                    }
                    composable( route = "Gallery") {
                        ContentGallery(
                            historial,
                            onRegister = { historial.add(it) })

                    }
                    composable( route = "Settings") {
                        ContentSettings(
                            historial,
                            onRegister = { historial.add(it) })

                    }
                }
            },
```

```kotlin
@OptIn( ...markerClass = ExperimentalMaterial3Api::class)
@Composable
fun Topbar(topBarTitle: String) {
    CenterAlignedTopAppBar(
        title = { Text(text = topBarTitle, color = Color.White) },
        colors = topAppBarColors(
            containerColor = paBackground,
            titleContentColor = paText
        )
    )
}

1 Usage
@OptIn( ...markerClass = ExperimentalMaterial3Api::class)
@Composable
fun BottomBar(
    navController: NavController,
    bottomItems: List<BottomItem>,
    topBarTitle: (String) -> Unit
) {
    NavigationBar(
        containerColor = paBackground,
        contentColor = paText
    ) {
        val navBackStackEntry by navController.currentBackStackEntryAsState()
        val currentRoute = navBackStackEntry?.destination?.route

        Log.d( tag = "ROUTE",  msg = "currentRoute = $currentRoute")
        bottomItems.forEach { item ->
            val selected = currentRoute == item.route
            NavigationBarItem(
                selected = selected,
                onClick = {
                    topBarTitle(item.label)
                    navController.navigate(item.route){
                        launchSingleTop = true
                        restoreState = true
                    }
                },
                icon = {
                    Icon(
                        painterResource(id = item.icon),
                        contentDescription = item.label,
                        tint = if (selected) paSelected else paText,   // 🔥 color dinámico
                        modifier = Modifier.size(if (selected) 32.dp else 24.dp)  // 🔥 tamaño dinámico
                    )
                },
                colors = NavigationBarItemDefaults.colors(
                    selectedIconColor = paSelected,
                    unselectedIconColor = paText,
                    selectedTextColor = paSelected,
                    unselectedTextColor = paText,
                    indicatorColor = Color.Transparent
                )
```

```kotlin
@Composable
fun ContentHome(
    historial: SnapshotStateList<String>,
    onRegister: (String) -> Unit
) {

    LazyColumn(
        modifier = Modifier
            .fillMaxSize()
            .background(color = paBackground)
            .padding(top = 16.dp)
    ) {}
}

//@Preview(showBackground = true)
1 Usage
@Composable
fun ContentAboutMe(
    historial: SnapshotStateList<String>,
    onRegister: (String) -> Unit
) {

    var counter by rememberSaveable { mutableIntStateOf( value = 0) }

    LazyColumn(
        modifier = Modifier
            .fillMaxSize()
            .background(color = paBackground)
```

```kotlin
@Composable
fun ContentGallery(
    historial: SnapshotStateList<String>,
    onRegister: (String) -> Unit
) {
    val images = listOf(R.drawable.im_characters_1, R.drawable.im_argi_1, R.drawable.im_argi_2,
        R.drawable.im_argi_3, R.drawable.im_grabi_1, R.drawable.im_grabi_2,
        R.drawable.im_grabi_3, R.drawable.im_gravi_e, R.drawable.im_suur_1)
    LazyVerticalGrid(
        columns = GridCells.Fixed( count = 3),
        modifier = Modifier
            .background( color = paBackground).fillMaxSize()
            .padding( all = 16.dp),
        horizontalArrangement = Arrangement.spacedBy(4.dp),
        verticalArrangement = Arrangement.spacedBy(4.dp)
    ) {

        items( count = images.size) { index ->
            Image(
                painter = painterResource(id = images[index]),
                contentDescription = "Imagen $index",
                modifier = Modifier
                    .size(72.dp)
                    .clip( shape = RoundedCornerShape( size = 12.dp)),
                contentScale = ContentScale.Crop
            )
        }
    }
}

1 Usage
@Composable
fun ContentSettings(
    historial: SnapshotStateList<String>,
    onRegister: (String) -> Unit
) {

    LazyColumn(
        modifier = Modifier
            .fillMaxSize()
            .background(color = paBackground)
            .padding(top = 16.dp)
    ) {}
}
```

Ejercicio02. Implementa 3 mejoras en el About Me. Comparte con el profesor la aplicación mediante Github. Explica en el README.md cuales son las 3 mejoras.

```kotlin
@SuppressLint( ...value = "UnusedMaterial3ScaffoldPaddingParameter")
@Composable
fun ViewContainer(navController: NavHostController) {
    val snackbarHostState = remember { SnackbarHostState() }
    val scope = rememberCoroutineScope()
    val historial = remember { mutableStateListOf<String>() }
    val bottomItems = listOf(
        BottomItem( route = "home", icon = R.drawable.ic_home, label = "Home", topIcon = R.drawable.ic_browse),
        BottomItem( route = "AboutMe", icon = R.drawable.ic_aboutme, label = "About Me", topIcon = R.drawable.ic_mail),
        BottomItem( route = "Gallery", icon = R.drawable.ic_gallery, label = "Gallery", topIcon = R.drawable.ic_share),
        BottomItem( route = "Settings", icon = R.drawable.ic_settings, label = "Settings", topIcon = null)
    )
    var topBarTitle by remember { mutableStateOf( value = "Home") }
    var topBarIcon by remember { mutableStateOf<Int?>( value = null) }
    var selectedImage by remember { mutableStateOf<Int?>( value = null) }
    val aboutMeVM: AboutMeViewModel = viewModel()
    var likeCounter by rememberSaveable { mutableIntStateOf( value = 0) }

    PersonalProfileApplicationTheme {
        Scaffold(
            snackbarHost = { SnackbarHost(snackbarHostState) },
            topBar = { Topbar(topBarTitle, topBarIcon, selectedImage) },
            bottomBar = {
                BottomBar(
                    navController = navController,
                    bottomItems = bottomItems,
                    topBarTitle = { newTitle -> topBarTitle = newTitle },
                    topBarIcon = { newIcon -> topBarIcon = newIcon }
                )
            },
            content = { padding ->
                NavHost(
                    navController = navController,
                    startDestination = "home",
                    modifier = Modifier.padding( paddingValues = padding)
                ) {
                    composable( route = "home") { ContentHome(historial, onRegister = { historial.add(it) }) }
                    composable( route = "AboutMe") {
                        ContentAboutMe(
                            historial,
                            onRegister = { historial.add(it) },
                            counter = likeCounter,
                            onIncrement = {likeCounter++})
                    }
                    composable( route = "Gallery") {
                        ContentGallery(
                            historial,
                            onRegister = { historial.add(it) },
                            onImageSelected = { newImage -> selectedImage = newImage }, viewModel()
                        )
                    }
                    composable( route = "Settings") {
                        ContentSettings(
                            historial,
                            onRegister = { historial.add(it) }, viewModel = aboutMeVM)

                    }
                }
            },
```

```kotlin
@OptIn( ...markerClass = ExperimentalMaterial3Api::class)
@Composable
fun Topbar(topBarTitle: String, icon: Int?, selectedImage: Int?) {
    val context = LocalContext.current
    CenterAlignedTopAppBar(
        title = { Text(text = topBarTitle, color = paText) },
        actions = {
            if (icon != null) {
                Icon(
                    painter = painterResource(id = icon),
                    contentDescription = null,
                    tint = paText,
                    modifier = Modifier
                        .size(36.dp)
                        .padding(end = 8.dp)
                        .clickable {
                            when (icon) {
                                R.drawable.ic_browse -> {
                                    val url = "https://www.google.es"
                                    val intent = Intent( action = Intent.ACTION_VIEW, Uri.parse( uriString = url))
                                    context.startActivity( intent = intent)
                                }

                                R.drawable.ic_mail -> {
                                    val intent = Intent( action = Intent.ACTION_SENDTO).apply {
                                        data = Uri.parse( uriString = "mailto:alumno@ejemplo.com")
                                        putExtra( name = Intent.EXTRA_SUBJECT,  value = "Saludos desde Compose")
                                        putExtra(
                                            name = Intent.EXTRA_TEXT,
                                            value = "Hola, este es un mensaje de prueba"
                                        )
                                    }

                                R.drawable.ic_share -> {
                                   selectedImage?.let { imageRes ->

                                       val uri = ShareUtils().getImageUriFromDrawable(context,  drawableId = imageRes)

                                       val intent = Intent( action = Intent.ACTION_SEND).apply {
                                           type = "image/*"
                                           putExtra( name = Intent.EXTRA_STREAM,  value = uri)
                                           addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION)
                                       }
                                       context.startActivity(
                                           intent = Intent.createChooser(
                                               target = intent,
                                               title = "Compartir con"
                                           )
                                       )
                                   }
                                }
                            }
                        }

                )
            }
        },
        colors = topAppBarColors(
            containerColor = paBackground,
            titleContentColor = paText
        )
```